

06. 자료구조와 알고리즘

13. 최단 경로 알고리즘

학습목표

- 다익스트라 알고리즘을 사용하여 그래프 내에서 최단 경로를 구할 수 있다.
- A* 알고리즘을 사용하여 그래프 내에서 최단 경로를 구할 수 있다.

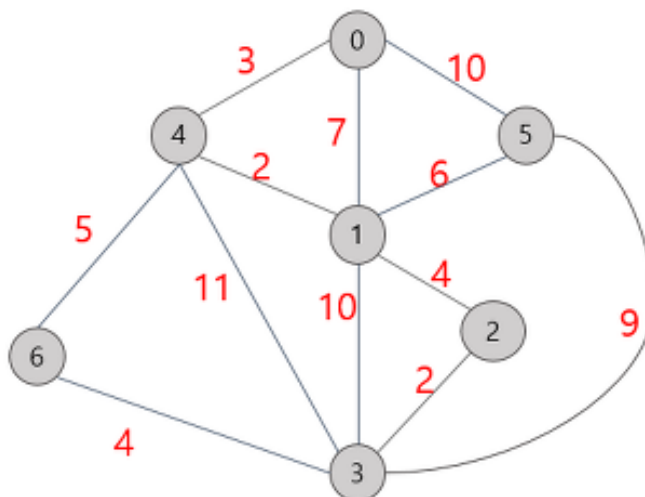
들어가며

최단 경로(Shortest Path)는 **가중치 그래프에서 두 정점을 연결하는 경로 중 가중치의 합이 최소인 경로**를 말한다. 이번 시간에는 최단 경로를 구하는 다익스트라 알고리즘과 이를 변형 시킨 A* 알고리즘에 대해서 알아보도록 하자.

다익스트라 알고리즘

다익스트라 알고리즘(Dijkstra Algorithm)은 **시작 정점에서 모든 다른 정점까지의 최단 경로**를 구할 수 있는 알고리즘이다. **최단 경로의 일부를 이루는 경로도 최단 경로**라는 점을 이용한 알고리즘으로 다음으로 이동해야 할 정점을 구할 때 가장 가까운 정점을 선택해서 추가한다.* 다익스트라 알고리즘으로 최단 경로를 구하기 위해서는 그래프에 **음수인 가중치가 있어서는 안 되고, 간선이 없을 경우 무한대 값으로 저장한다**. 아래의 그래프를 이용해 알고리즘을 배워보자.

* 이를 그때그때 최적해를 고른다 하여 **그리디 알고리즘(Greedy Algorithm)**이라고 한다.



의사코드

의사코드는 아래와 같다.

```
def dijkstra(graph, source):  
    # 1. source로부터의 최단 경로의 길이를 저장하는 dist 배열 생성  
    for vertex in graph.vertex:  
        dist[vertex] = Inf  
  
    # 2. 경로를 찾았다는 것을 기록할 isVisited 배열을 생성한다.  
    for vertex in graph.vertex:  
        isVisited[vertex] = False  
  
    # 3. 초기 값을 세팅한다.  
    dist[source] = 0  
  
    # 4. 모든 최단 경로를 찾을 때까지 반복한다.  
    while (isFound == False):  
        # 4-1. 방문하지 않은 정점 중 dist가 최소인 정점을 찾는다.  
        v = isVisited[v] == False && dist[v]가 최소인 정점  
  
        # 4-2. 방문을 표시한다.  
        isVisited[v] = True  
  
        # 4-3. dist를 업데이트 한다.  
        for neighbor in graph.neighbor(v):  
            dist[neighbor] = min(dist[negibor], dist[v] + graph.edge(v, neighbor))  
  
    return dist
```

구현

C#으로 구현하면 아래와 같다.

```
class Dijkstra
{
    // 연결되지 않은 간선을 포함하기 위한 무한대 값이다. 임의로 설정해준다.
    const int Inf = 987654321;

    // 그래프를 구성한다. 그래프는 교안을 참고한다.
    // 인접 행렬 방식을 사용한다.
    private static int[][] graph;
    static void ConstructGraph()
    {
        graph = new int[7][];

        graph[0] = new int[] { 0, 7, Inf, Inf, 3, 10, Inf };
        graph[1] = new int[] { 7, 0, 4, 10, 2, 6, Inf };
        graph[2] = new int[] { Inf, 4, 0, 2, Inf, Inf, Inf };
        graph[3] = new int[] { Inf, 10, 2, 0, 11, 9, 4 };
        graph[4] = new int[] { 3, 2, Inf, 11, 0, Inf, 5 };
        graph[5] = new int[] { 10, 6, Inf, 9, Inf, 0, Inf };
        graph[6] = new int[] { Inf, Inf, Inf, 4, 5, Inf, 0 };
    }

    // start 노드에서 end까지의 최단 거리를 구한다.
    static int GetDistance(int start, int end)
    {
        // dist 배열을 만든다.

        // ↳ dist[v] : dist -> v까지의 최단 거리
    }
}
```

```

int[] dist = new int[7];

for (int i = 0; i < 7; ++i)
{
    dist[i] = Inf;
}

dist[start] = 0;

// 최단 경로를 빠르게 찾기 위해 우선순위 큐를 사용한다.
// PriorityQueue<정점, 거리>
PriorityQueue<int, int> pq = new PriorityQueue<int, int>();
pq.Enqueue(start, dist[start]);

// 모든 정점에 대한 최단 경로를 찾을 때까지 반복한다.
while (pq.Count > 0)
{
    // 현재 방문할 정점을 우선순위 큐에서 가져온다.
    int node = pq.Dequeue();

    // 새로운 정점을 방문했기 때문에 dist를 업데이트한다.
    for (int next = 0; next < 7; ++next)
    {
        // start -> node -> next로 가는 거리를 구해본다.
        int distViaNode = dist[node] + graph[node][next];

        // node를 경유해서 next로 가는 것이 더 빠른지 확인한다.
        // 즉, 새로운 최단 거리를 찾았는지 검사한다.
        if (dist[next] > distViaNode)
        {
            dist[next] = distViaNode;
            pq.Enqueue(next, dist[next]);
        }
    }
}

```

```

    }

    }

    return dist[end];
}

static void Main()
{
    ConstructGraph();

    Console.WriteLine(GetDistance(0, 3)); // 11
}
}

```

A* 알고리즘

다익스트라 알고리즘은 현실에 적용하기가 부담스럽다. 예를 들어 네비게이션 프로그램을 구현하기 위해 다익스트라 알고리즘을 사용한다고 해보자. 다익스트라 알고리즘을 사용하면 실시간으로 달라지는 교통량을 고려할 수 없다. 교통량이 달라질 때마다 간선의 가중치를 변화시킬 수 없기 때문이다.

A* 알고리즘은 **다익스트라 알고리즘과 동일한 로직을 사용하나, 최단 경로를 찾기 위한 가중치 계산 부분이 다르다.** A* 알고리즘은 **휴리스틱 접근법**(Heuristic Approach)을 사용한다. 휴리스틱은 쉽게 말해 **직관**을 의미한다. 현재 상태의 비용을 $g(x)$, 현재 상태에서 다음 상태로 이동할 때의 휴리스틱 함수를 $h(x)$ 라 할 때, 둘을 더한 $f(x) = g(x) + h(x)$ 로 가중치를 계산한다. 수식에서 볼 수 있듯 **휴리스틱에 따라 A* 알고리즘의 품질이 매우 달라지는 데**, 휴리스틱이 반드시 **허용 가능**(Admissible)해야 한다. 보통 맨해튼 거리(Manhattan Distance)나 유클리디언 거리(Euclidean Distance)를 사용한다.* 만약 $h(x) = 0$ 이면 다익스트라 알고리즘과 동일하다.

* [맨해튼 거리와 유클리디언 거리](#)

구현

A* 알고리즘을 직접 구현해보자. 시작점은 S, 목적지는 G, 벽은 #으로 표현한다.

```

class AStar
{

```

```

const int MAX_Y = 10;

const int MAX_X = 10;

static char[][] map = new char[MAX_Y][];

// 맵을 구성한다.
static void ConstructMap()
{
    map[0] = "      ".ToCharArray();
    map[1] = "      ".ToCharArray();
    map[2] = "      ".ToCharArray();
    map[3] = "  #   ".ToCharArray();
    map[4] = " S # G ".ToCharArray();
    map[5] = "  #   ".ToCharArray();
    map[6] = "      ".ToCharArray();
    map[7] = "      ".ToCharArray();
    map[8] = "      ".ToCharArray();
    map[9] = "      ".ToCharArray();
}

static void PrintMap()
{
    foreach (char[] row in map)
    {
        Console.WriteLine(row);
    }
}

// 시작 지점과 목적 지점을 찾는다.
static int startX, startY, endX, endY;

static void FindStartAndEnd()
{

```

```

for (int y = 0; y < MAX_Y; ++y)
{
    for (int x = 0; x < MAX_X; ++x)
    {
        if (map[y][x] == 'S')
        {
            startX = x;
            startY = y;
        }
        else if (map[y][x] == 'G')
        {
            endX = x;
            endY = y;
        }
    }
}
}

```

// A* 알고리즘을 위한 노드

```

class AStarNode
{
    public int X;
    public int Y;
    public int F;
    public AStarNode Path;
}

```

// (x1, y1) -> (x2, y2) 가기 위한 휴리스틱 함수

// 맨해튼 거리

```

static int Heuristic(int x1, int y1, int x2, int y2)

```

```

{

    int dx = Math.Abs(x1 - x2);

    int dy = Math.Abs(y1 - y2);


    return 10 * (dx + dy);
}


const int Inf = 987654321;


// A* 알고리즘으로 최단 경로를 찾는다.
static void SetPath()
{
    // 시작 지점과 목적 지점을 찾는다.
    FindStartAndEnd();


    // 경로를 구성한다.
    // path[y, x] : map[y][x]에 대응되는 AStarNode
    AStarNode[,] path = new AStarNode[MAX_Y, MAX_X];
    for (int y = 0; y < MAX_Y; ++y)
    {
        for (int x = 0; x < MAX_X; ++x)
        {
            path[y, x] = new AStarNode() { X = x, Y = y, F = Inf };
        }
    }


    // 우선순위 큐를 생성한다.

```



```

// PriorityQueue<AStarNode, F값>
PriorityQueue<AStarNode, int> pq = new PriorityQueue<AStarNode, int>();
pq.Enqueue(path[startY, startX], path[startY, startX].F);

// 8방향을 탐색한다.
//      ↖ ↑ ↗ → ↘ ↓ ↙ ←
int[] dy = { -1, -1, -1, 0, 1, 1, 1, 0 };
int[] dx = { -1, 0, 1, 1, 1, 0, -1, -1 };
int[] dg = { 14, 10, 14, 10, 14, 10, 14, 10 }; // 다음 노드로 이동하기 위한 비용

// 경로를 찾을 때까지 반복한다.
while (pq.Count > 0)
{
    // 방문할 노드를 가져온다.
    AStarNode node = pq.Dequeue();

    // 목적지에 도달했는지 확인한다.
    if (node.X == endX && node.Y == endY)
    {
        break;
    }

    // 8방향으로 탐색을 진행한다.
    for (int i = 0; i < 8; ++i)
    {
        int nx = node.X + dx[i];
        int ny = node.Y + dy[i];
    }
}

```

```

// 인덱스 유효성을 검사한다.
if (nx < 0 || nx >= MAX_X || ny < 0 || ny >= MAX_Y)
{
    continue;
}

// 이동 가능한지 검사한다.
if (map[ny][nx] == '#')
{
    continue;
}

// F값을 계산한다.
//  $f(x) = g(x) + h(x)$ 
int f = dg[i] + Heuristic(nx, ny, endX, endY);

// 새로운 부분 최단 경로를 찾았는지 확인한다.
AStarNode newNode = path[ny, nx];
if (newNode.F > f)
{
    newNode.F = f;
    newNode.Path = node;
    pq.Enqueue(newNode, newNode.F);
}
}
}

```

```

// 경로를 맵에 표시한다.
AStarNode current = path[endY, endX].Path;
while (true)
{
    if (current.X == startX && current.Y == startY)
    {
        break;
    }

    map[current.Y][current.X] = '*';
    current = current.Path;
}

static void Main()
{
    ConstructMap();
    SetPath();
    PrintMap();
}
}

```

더 나아가기

1. 수업 내용을 복기하면서 아래 내용을 정리해보세요.
 - a. 현재의 다익스트라 알고리즘은 최단 거리는 알 수 있지만 어떻게 이동해야 하는지는 알려주지 않습니다. 조금만 응용하면 경로를 구할 수 있습니다. 경로를 구할 수 있게 코드를 수정해 보세요.
 - b. 다익스트라 알고리즘을 사용할 때 가중치에 음수가 있으면 안 되는 이유를 논리적으로 설명해 보세요.

- c. 다익스트라 알고리즘과 A* 알고리즘의 차이는 무엇인가요?
2. 음의 간선이 존재해도 최단 경로를 구할 수 있는 벨만-포드 알고리즘에 대해서 조사해보세요. (자료 [1](#), [2](#))
3. 모든 정점의 최단 경로를 구할 수 있는 플로이드-와샬 알고리즘에 대해서 조사해보세요. (자료 [1](#))
4. 아래 문제를 풀어보며 최단 경로 알고리즘을 응용해봅시다.
- a. [1753번: 최단경로](#)
 - b. [1504번: 특정한 최단 경로](#)
 - c. [13549번: 숨바꼭질 3](#)
 - d. [9370번: 미확인 도착지](#)

참고자료

- [C로 배우는 쉬운 자료구조](#)
- [누구나 자료 구조와 알고리즘](#)
- [〈Dijkstra의 최단 경로 알고리즘〉 기본개념과 알고리즘 - waca's field](#)